

BASES DE DATOS

ENRIQUE REINOSA - CALIXTO MALDONADO - ROBERTO MUÑOZ -
LUIS DAMIANO - MAXIMILIANO ABRUTSKY



Alfaomega



BASES DE DATOS

BASES DE DATOS

Enrique José Reinoso - Calixto Alejandro Maldonado - Roberto Muñoz -
Luis Esteban Damiano - Maximiliano Adrián Abrutsky



Buenos Aires • Bogotá • México DF • Santiago de Chile

Reinosa, Enrique José

Bases de Datos / Enrique José Reinosa ; Calixto Alejandro Maldonado ; Roberto Muñoz ; Luis Esteban Damiano ; Maximiliano Adrián Abrutsky. - 1a ed.
- Buenos Aires : Alfaomega Grupo Editor Argentino, 2012.
384 p. ; 24x21 cm.

ISBN 978-987-1609-31-4

1. Informática. 2. Base de datos. I. Maldonado, Calixto II. Muñoz, Roberto
III. Título
CDD 005.3

Queda prohibida la reproducción total o parcial de esta obra, su tratamiento informático y/o la transmisión por cualquier otra forma o medio sin autorización escrita de Alfaomega Grupo Editor Argentino S.A.

Edición: Damián Fernández

Corrección de estilo: Juan Arana, Silvia Mellino y Vanesa García

Diagramación de interiores: Diego Linares y Patricia Baggio

Revisión de armado: Juan Micán

Revisión técnica: Claudia Deco, Cristina Bendec, Calixto Alejandro Maldonado y Roberto Muñoz

Diseño de tapa: Diego Linares

Internet: <http://www.alfaomega.com.mx>

Todos los derechos reservados © 2012, por Alfaomega Grupo Editor Argentino S.A.
Paraguay 1307, PB, oficina 11

ISBN 978-987-1609-31-4

Queda hecho el depósito que prevé la ley 11.723

NOTA IMPORTANTE: La información contenida en esta obra tiene un fin exclusivamente didáctico y, por lo tanto, no está previsto su aprovechamiento a nivel profesional o industrial. Las indicaciones técnicas y programas incluidos han sido elaborados con gran cuidado por el autor y reproducidos bajo estrictas normas de control. Alfaomega Grupo Editor Argentino S.A. no será jurídicamente responsable por: errores u omisiones; daños y perjuicios que se pudieran atribuir al uso de la información comprendida en este libro, ni por la utilización indebida que pudiera dársele.

Los nombres comerciales que aparecen en este libro son marcas registradas de sus propietarios y se mencionan únicamente con fines didácticos, por lo que Alfaomega Grupo Editor Argentino S.A. no asume ninguna responsabilidad por el uso que se dé a esta información, ya que no infringe ningún derecho de registro de marca. Los datos de los ejemplos y pantallas son ficticios, a no ser que se especifique lo contrario.

Los hipervínculos a los que se hace referencia no son necesariamente administrados por la editorial, por lo que no somos responsables de sus contenidos o de su disponibilidad en línea.

Empresas del grupo:

Argentina: Alfaomega Grupo Editor Argentino, S.A.

Paraguay 1307 P.B. "11", Buenos Aires, Argentina, C.P. 1057

Tel.: (54-11) 4811-7183 / 0887

E-mail: ventas@alfaomegaeditor.com.ar

México: Alfaomega Grupo Editor, S.A. de C.V.

Pitágoras 1139, Col. Del Valle, México, D.F., México, C.P. 03100

Tel.: (52-55) 5575-5022 – Fax: (52-55) 5575-2420 / 2490. Sin costo: 01-800-020-4396

E-mail: atencionalcliente@alfaomega.com.mx

Colombia: Alfaomega Colombiana S.A.

Carrera 15 No. 64 A 29, Bogotá, Colombia

PBX (57-1) 2100122 – Fax: (57-1) 6068648

E-mail: cliente@alfaomega.com.co

Chile: Alfaomega Grupo Editor, S.A.

Doctor La Sierra 1437 – Providencia, Santiago, Chile

Tel.: (56-2) 235-4248 – Fax: (56-2) 235-5786

E-mail: agechile@alfaomega.cl

“Dedicado a mi familia por el tiempo restado a ellos para la generación de este libro y a mis compañeros docentes y ayudantes por su colaboración en el proceso educativo de nuestros alumnos”.

Enrique José Reinosa

“A Alicia, mi compañera por veinte años y a Ludmila, Emilio y Sofía, mis tesoros, les dedico el esfuerzo y el resultado”.

Calixto Alejandro Maldonado

“A mis padres, por inculcarme el valor del estudio. A Eugenia, Alfonsina y Amparo por el estímulo permanente en todo lo que emprendo”.

Roberto Muñoz

“A los intelectualmente generosos, quienes ayudan a la construcción de un conocimiento para todos”.

“A mi familia, a Valentina, Bruno y Andrea”.

Luis Esteban Damiano

“A Kela, mi madre, por inculcarme el esfuerzo y sus principios morales que me conducen en la vida, a Marcos, hermano y mejor amigo, y a Roberto un ejemplo de profesional y persona, sin ellos este aporte no hubiese sido posible”.

Maximiliano Adrián Abrutsky

Mensaje del Editor

Los conocimientos son esenciales en el desempeño profesional. Sin ellos es imposible lograr las habilidades para competir laboralmente. La universidad o las instituciones de formación para el trabajo ofrecen la oportunidad de adquirir conocimientos que serán aprovechados más adelante en beneficio propio y de la sociedad. El avance de la ciencia y de la técnica hace necesario actualizar continuamente esos conocimientos. Cuando se toma la decisión de embarcarse en una vida profesional, se adquiere un compromiso de por vida: mantenerse al día en los conocimientos del área u oficio que se ha decidido desempeñar.

Alfaomega tiene por misión ofrecerles a estudiantes y profesionales conocimientos actualizados dentro de lineamientos pedagógicos que faciliten su utilización y permitan desarrollar las competencias requeridas por una profesión determinada. Alfaomega espera ser su compañera profesional en este viaje de por vida por el mundo del conocimiento.

Alfaomega hace uso de los medios impresos tradicionales en combinación con las tecnologías de la información y las comunicaciones (IT) para facilitar el aprendizaje. Libros como éste tienen su complemento en una página Web, en donde el alumno y su profesor encontrarán materiales adicionales, información actualizada, pruebas (test) de autoevaluación, diapositivas y vínculos con otros sitios Web relacionados.

Esta obra contiene numerosos gráficos, cuadros y otros recursos para despertar el interés del estudiante, y facilitarle la comprensión y apropiación del conocimiento.

Cada capítulo se desarrolla con argumentos presentados en forma sencilla y estructurada claramente hacia los objetivos y metas propuestas. Cada capítulo concluye con diversas actividades pedagógicas para asegurar la asimilación del conocimiento y su extensión y actualización futuras.

Los libros de Alfaomega están diseñados para ser utilizados dentro de los procesos de enseñanza-aprendizaje, y pueden ser usados como textos guía en diversos cursos o como apoyo para reforzar el desarrollo profesional.

Alfaomega espera contribuir así a la formación y el desarrollo de profesionales exitosos para beneficio de la sociedad.

Enrique José Reinoso

- Analista Universitario de Sistemas de la UTN – FRBA.
- Ingeniero en Sistemas de Información de la UTN – FRBA.
- Posgrado en Ingeniería Gerencial (MBA) en la UTN – FRBA.
- Docente Investigador Categorizado en la UTN – FRBA.
- Profesor Titular Ordinario y Jefe de las Cátedras de:
 - Gestión de Datos
 - Paradigmas de Programación
 - Técnicas Gráficas por Computadora
 - Técnicas Avanzadas de Programación
 - Tecnologías Avanzadas en la Construcción de Software
 - Administración de Recursos
- Coautor del Plan de Estudios 1995 de la Carrera Ingeniería en Sistemas de Información de la UTN.
- Profesor Tutor de Tesis de Títulos Intermedios de Analista Universitario de Sistemas.
- Director de Desarrollo de Software de la UTN – FRBA.
- Director de Desarrollo de Software del Instituto Nacional de Educación Técnica (INET).
- Consultor Independiente en Desarrollo de Software y Bases de Datos.

Calixto Alejandro Maldonado

- Ingeniero en Sistemas de Información de la UTN-FRC.
- Profesional certificado Oracle Developer y DBA-OCP.
- Doctorando de Ingeniería en software basado en componentes reutilizables, en la Universidad de Vigo.
- Docente e Investigador en la UTN-FRC y la Universidad Empresarial 21.
- Consultor DBA independiente.

Roberto Muñoz

- Ingeniero en Sistemas de Información de la UTN-FRC.
- Especialista en Docencia Universitaria de la UTN-FRC.
- Maestrando en Ingeniería en Sistemas de Información de la UTN-FRC.
- Jefe de la cátedra de Gestión de Datos de la UTN-FRC.
- Investigador en la UTN-FRC.
- Coordinador de la Diplomatura Superior en Administración y Exploración de Bases de Datos en la UTN-FRC.
- Coordinador universitario del Programa Nacional “Plan +” de “Becas Control-F” en la UTN-FRC.
- Integrante del Centro de Investigación y Desarrollo de Sistemas (CIDS) en la UTN-FRC.

Luis Esteban Damiano

- Analista de Sistemas de Computación de la UBP-ISP, Argentina.
- Licenciado en Tecnología Educativa de la UTN-FRC.
- Docente de las cátedras de Gestión de datos y Programación de aplicaciones visuales I en la UTN-FRC.
- Analista de Sistemas en la Dirección de Sistemas de Gobierno en Córdoba, Argentina.
- Capacitador del Programa Nacional “Plan+” de “Becas Control-F” (años 2008, 2009, 2010).
- Diseñador de sistemas independiente.

Maximiliano Adrián Abrutsky

- Ingeniero en Sistemas de Información de la UTN-FRC.
- Docente, Investigador y Maestrando (desarrollando Tesis) en la UTN-FRC.
- Docente en la UNC - Fac. Cs. Económicas.
- Socio gerente en Liricus Soluciones Informáticas.
- Java Developer Senior - DBA certificado DB2 - SQL Server.

Revisores técnicos:

Claudia Deco:

- Licenciada en Matemáticas de la Universidad Nacional de Rosario, Argentina.
- Magíster en Informática de la Universidad de la República, Uruguay.
- Doctora en Ingeniería de la Universidad Nacional de Rosario, Argentina.

Cristina Bendec:

- Ingeniera Electrónica de la Universidad de Rosario, Argentina.
- Magíster en Informática de la Universidad de la República, Uruguay.
- Especialista en Gestión de Sistemas de Información de la Universidad Católica Argentina.

Calixto Alejandro Maldonado

Roberto Muñoz

Agradecemos las sugerencias y comentarios realizados por:

- Lucila Patricia Arellano Mendoza, Departamento de Computación de la Facultad de Ingeniería de la Universidad Nacional Autónoma de México.
- Nancy Ocotitla Rojas, Docente del Departamento de Ingeniería en Sistemas Computacionales en la ESCOM – IPN.
- Euler Hernández Contreras, Jefe de la asignatura de Bases de Datos de la carrera de Ingeniería en Sistemas en la Escuela Superior de Cómputo del Instituto Politécnico Nacional.

Contenido

Prólogo	XIX	2.1.10 Reglas de integridad	36
Capítulo 1		2.1.11 Diccionario de datos	37
Estructura y tipos de bases de datos	1	2.1.12 Diseño de software utilizando bases de datos relacionales	38
1.1 Introducción	2	2.1.13 El acceso a las bases de datos con aplicaciones escritas en JAVA	39
1.2 Definición de base de datos	3	2.1.14 Acceso a las bases de datos a través de servicios Web	40
1.3 Sistema de Gestión de Bases de Datos	4	2.2 Álgebra relacional	41
1.4 Usuarios de la base de datos	5	2.2.1 Operaciones de conjuntos	44
1.5 Seguridad de las bases de datos	7	2.2.2 Operaciones relacionales	48
1.5.1 Creación de usuarios	8	2.2.3 Reunión	50
1.5.2 Otorgar privilegios de sistema	8	2.2.4 División	54
1.5.3 Usar roles para administrar los accesos de base de datos	9	2.2.5 Renombrar	55
1.5.4 Otorgar privilegios de objetos	10	2.2.6 Funciones	56
1.5.5 El cambio de contraseñas	11	2.2.7 Agrupación	57
1.5.6 Uso de sinónimos para la transparencia de base de datos	11	2.2.8 Relaciones temporales	58
1.6 Funciones y responsabilidades de un DBA	12	2.2.9 Cálculo relacional	61
1.7 Arquitectura ANSI/SPARC	13	2.3 Normalización	63
1.8 Modelos de datos	17	2.3.1 Fundamentos de la normalización	63
1.8.1 Modelo de datos orientado a objetos	17	2.3.2 A qué se refiere la normalización	64
1.8.2 Modelos de datos basados en registros	19	2.3.3 A qué no se refiere la normalización	65
1.9 Resumen	23	2.3.4 Normalización aspecto formal	66
1.10 Contenido de la página Web de apoyo	23	2.3.5 Concepto de normalizar	66
1.10.1 Mapa conceptual del capítulo	23	2.3.6 Objetivos de la normalización	67
1.10.2 Autoevaluación	23	2.4 Origen de los datos	67
1.10.3 Presentaciones	23	2.4.1 Los datos	67
Capítulo 2		2.4.2 Problemática asociada al origen de datos	68
Modelo de datos relacional	25	2.4.3 Los resultados	69
2.1 Introducción	26	2.5 Las formas normales	69
2.1.1 Conceptos básicos en el modelo relacional	27	2.5.1 las normas	69
2.1.2 Propiedades de las relaciones	29	2.5.2 Dominio (extensión conceptual)	70
2.1.3 Reglas de Codd	29	2.5.3 Fundamento teórico de las normas	70
2.1.4 Atributos claves	31	2.5.4 Enumeración de las normas	72
2.1.5 Condiciones de las claves candidatas	31	2.5.5 Interpretación y aplicación de las formas normales	73
2.1.6 Claves primarias	32	2.6 Las estructuras	92
2.1.7 Clave foránea	33	2.6.1 El modelo y sus estructuras	92
2.1.8 Marcador de valores desconocidos	34	2.7 Un caso de estudio	99
2.1.9 Correspondencias	35	2.8 Resumen	101
		2.9 Contenido de la página Web de apoyo	101
		2.9.1 Mapa conceptual del capítulo	101
		2.9.2 Autoevaluación	101
		2.9.3 Presentaciones*	101

Capítulo 3

SQL	103
3.1 Introducción	104
3.2 Algo de historia del lenguaje SQL	104
3.3 Características del lenguaje SQL	105
3.4 El lenguaje SQL y su sublenguaje de definición de datos o DDL	106
3.5 Sentencias del sublenguaje DML	112
3.5.1 Funciones de fila simple	113
3.5.2 Funciones numéricas	113
3.5.3 Funciones generales de comparación	115
3.5.4 Funciones de conversión	115
3.5.5 Funciones de grupo	115
3.5.6 La cláusula FROM	118
3.5.7 La cláusula WHERE	121
3.5.8 La cláusula GROUP BY	122
3.5.9 La cláusula HAVING	122
3.5.10 La cláusula ORDER BY	123
3.6 Sentencias del sublenguaje DML. INSERT, UPDATE, DELETE	123
3.6.1 Sentencia INSERT	123
3.6.2 Sentencia UPDATE	124
3.6.3 Sentencia MERGE	124
3.6.4 Sentencia DELETE	125
3.7 Sentencias del sublenguaje TCL de control de transacciones	125
3.8 Procesamiento de consultas	127
3.8.1 Plan de consultas	128
3.8.2 Optimización de consultas	129
3.9 Resumen	129
3.10 Contenido de la página Web de apoyo	129
3.10.1 Mapa conceptual del capítulo	129
3.10.2 Autoevaluación	129
3.10.3 Presentaciones*	129

Capítulo 4

Lenguaje procedimental como extensión de SQL	131
4.1 Introducción	132
4.2 Vista general de PL/SQL	132
4.3 Uso del PL/SQL para acceder a la base de datos Oracle	132
4.4 Programas con PL/SQL	133
4.4.1 Modularidad	133
4.4.2 Procedimientos, funciones, triggers y paquetes	133
4.5 Componentes de un bloque PL/SQL	134
4.5.1 Las construcciones lógicas y de control de flujo	135

4.6 Cursores	135
4.7 Errores	136
4.8 El desarrollo de un bloque PL/SQL	136
4.8.1 Los tipos de datos de la base de datos	136
4.8.2 Tipos de datos que son propios del lenguaje PL/SQL	137
4.9 Interacción con la base de datos Oracle	138
4.10 El tratamiento de transacciones en PL/SQL	139
4.11 Sentencias de control de flujo	141
4.11.1 Usando loops	142
4.12 Manejo de cursores	144
4.13 Manejo de errores	148
4.13.1 Excepciones comunes	150
4.13.2 Codificando en la sección de excepciones	150
4.14 Construyendo procedimientos y funciones con PL/SQL	151
4.14.1 Creando paquetes con PL/SQL	155
4.14.2 Creando triggers con PL/SQL	157
4.15 Resumen	159
4.16 Contenido de la página Web de apoyo	159
4.16.1 Mapa conceptual del capítulo	159
4.16.2 Autoevaluación	159
4.16.3 Presentaciones	159

Capítulo 5

Bases de datos multidimensionales y tecnologías OLAP ...	161
5.1 Introducción	162
5.2 Bases de datos multidimensionales	162
5.2.1 Evolución de las bases de datos	162
5.2.2 Concepto	164
5.2.3 Estructura de almacenamiento	166
5.2.4 Dispersión de datos	167
5.3 Tecnologías OLAP	169
5.3.1 Introducción	169
5.3.2 Concepto de OLAP	169
5.3.3 Características de OLAP	169
5.3.4 Comparación entre el modelo OLTP y el modelo OLAP	170
5.4 Integración entre bases de datos y herramientas OLAP.	173
5.5 Arquitecturas OLAP y OLTP	174
5.6 OLAP: multidimensional contra relacional	175
5.6.1 OLAP multidimensional (MOLAP)	177
5.6.2 OLAP relacional (ROLAP)	178
5.6.3 OLAP híbrido (HOLAP)	182
5.7 Evaluación de servidores y herramientas OLAP	182
5.7.1 Características y funciones	182

5.7.2 Motores de servicios OLAP	182
5.7.3 Administración	183
5.7.4 Arquitectura global	183
5.8 Desarrollo de aplicaciones OLAP	184
5.9 Áreas de aplicación de las tecnologías OLAP	185
5.9.1 Análisis de ventas	185
5.9.2 Gestión de informes financieros	186
5.10 Ventajas y desventajas de OLAP	186
5.11 Resumen	187
5.12 Contenido de la página Web de apoyo	188
5.12.1 Mapa conceptual del capítulo	188
5.12.2 Autoevaluación	188
5.12.3 Presentaciones	188

Capítulo 6

Almacén de datos	189
6.1 Introducción	190
6.2 Inteligencia de negocio	191
6.2.1 Datos, Información y Conocimiento	192
6.3 Sistemas de almacén de datos	193
6.4 Concepto de almacén de datos	194
6.5 Características de un almacén de datos	194
6.6 Arquitectura del almacén de datos	196
6.6.1 Bases de datos fuentes	196
6.6.2 Base de datos con datos resumidos	197
6.6.3 Interfaces orientadas al usuario	197
6.7 Funcionalidades y objetivo	197
6.7.1 Acceso a fuentes	198
6.7.2 Carga	198
6.7.3 Almacenamiento	199
6.7.4 Consultas	199
6.7.5 Metadatos	199
6.8 Almacén de datos y Data Mart	200
6.9 La integridad de los datos	202
6.9.1 Concepto de Integridad	203
6.9.2 La perspectiva del usuario final	203
6.9.3 La perspectiva del Sistema de Información	203
6.9.4 Controles de integridad de datos	203
6.10 Costos y valor del almacén de datos	208
6.10.1 Costos de un almacén de datos	208
6.10.2 Valor del almacén de datos	210
6.10.3 Balance entre los costos y el valor	210
6.11 Impactos de la implementación de un almacén de datos	210
6.11.1 Recursos Humanos	211
6.11.2 Impactos organizacionales	211

6.11.3 Impactos técnicos del almacén de datos	212
6.11.4 Consideraciones finales	212
6.12 Estrategia recomendada para la implementación de un almacén de datos	213
6.12.1 Prototipo	213
6.12.2 Piloto	213
6.12.3 Prueba del concepto tecnológico	214
6.12.4 Arquitectura de un almacén de datos	214
6.12.5 Acceso a datos de usuario finales	215
6.12.6 Factores de riesgo	215
6.13 Resumen	216
6.14 Contenido de la página Web de apoyo	216
6.14.1 Mapa conceptual del capítulo	216
6.14.2 Autoevaluación	216
6.14.3 Presentaciones*	216

Capítulo 7

Minería de datos	217
7.1 Introducción	218
7.2 Concepto	218
7.3 Características de la minería de datos	219
7.4 Capacidades de la minería de datos	220
7.5 Herramientas algorítmicas de la minería de datos	221
7.5.1 Redes neuronales artificiales	222
7.5.2 Algoritmos Genéticos	224
7.5.3 Árboles de decisión	226
7.6 Modelado de la minería de datos	228
7.7 Integración entre almacén de datos y minería de datos	229
7.8 Ventajas de la minería de datos	230
7.9 Diferencias entre el análisis estadístico y la minería de datos	230
7.10 Aplicaciones de la minería de datos	232
7.11 Resumen	233
7.12 Contenido de la página Web de apoyo	233
7.12.1 Mapa conceptual del capítulo	233
7.12.2 Autoevaluación	233
7.12.3 Presentaciones	233

Capítulo 8

Bases de Datos Orientadas a Objetos	235
8.1 Introducción	236
8.2 Historia y origen de las BDOO	237
8.3 Conceptos fundamentales	238
8.4 Bases de la orientación a objetos	239

8.5 Características de las Bases de Datos Orientadas a Objetos	240
8.5.1 Modelo conceptual	240
8.5.2 Modelo de datos orientado a objetos	241
8.5.3 Persistencia de los datos	241
8.5.4 Almacenamiento y acceso de los objetos persistentes en una BDOO	242
8.5.5 Manifiesto de Atkinson	242
8.5.6 Intento de estandarización ODMG	244
8.5.7 Enfoques para la construcción de BDOO	251
8.6 Sistema de gestión de BDOO (SGBDOO)	252
8.6.1 Concepto de un SGBDOO	252
8.6.2 Objetivo	253
8.6.3 Características de los SGBDOO	253
8.6.4 Estructura de un SGBDOO	255
8.7 Rendimiento de las BDOO	256
8.8 Ventajas de las BDOO	256
8.9 Desventajas de las BDOO	257
8.10 Bases de Datos Objeto-Relacionales	258
8.10.1 Concepto	258
8.10.2 Características de las BDOR	258
8.10.3 Implementación en Oracle	259
8.11 Resumen	273
8.12 Contenido de la página Web de apoyo	273
8.12.1 Mapa conceptual del capítulo	273
8.12.2 Autoevaluación	273
8.12.3 Presentaciones	273

Capítulo 9

Implementando el modelo en Oracle Express Edition XE ..275

9.1. Introducción	276
9.1.1 Obteniendo el software e inicializando la instalación del Motor Oracle 10g XE	276
9.1.2. Descarga	276
9.1.3 Instalación	277
9.2 Creando y probando nuestro modelo de Estudiante – Universidad	283
9.3 Creación de las tablas y sus relaciones	285
9.3.1 Ejecución de script	285
9.3.2 Modelo de datos	285
9.3.3 Script (sentencias DDL)	286
9.3.4 Datos de prueba	290
9.3.5 Función escalar definida por el usuario	292
9.3.6 Procedimiento almacenado definido por el usuario	293
9.3.7 Ejemplos de ejecución	295

9.4 Contenido de la página Web de apoyo	296
9.4.1 Mapa conceptual del capítulo	296
9.4.2 Autoevaluación	296
9.4.3 Presentaciones	296

Capítulo 10

Implementando el modelo en IBM DB2

10.1 Introducción	298
10.2 Instalando e inicializando el servidor	298
10.2.1 Download	298
10.2.2 Instalación	298
10.3 Creando y probando nuestro modelo de Estudiante – Universidad	299
10.3.1 Creación de la base de datos	299
10.3.2 Creación de las tablas y sus relaciones	303
10.3.3 Modelo de datos	304
10.3.4 Datos de prueba	308
10.4 Contenido de la página Web de apoyo	310
10.4.1 Mapa conceptual del capítulo	310
10.4.2 Autoevaluación	310
10.4.3 Presentaciones	310

Capítulo 11

Implementando el modelo en SQL Server 2005

11.1 Introducción	312
11.2 Inicializando el servidor	312
11.2.1 Download	312
11.2.2 Instalación	312
11.2.3 Inicialización	316
11.3 Inicializando el cliente	317
11.3.1 Download	317
11.3.2 Instalación	317
11.3.3 Utilizando el SSMS	317
11.4 Creando y probando nuestro modelo de Estudiante–Universidad	318
11.4.1 Creación de la base de datos	319
11.4.2 Creación de las tablas y sus relaciones	320
11.4.3 Programación de la base de datos	327
11.5 Contenido de la página Web de apoyo	334
11.5.1 Mapa conceptual del capítulo	334
11.5.2 Autoevaluación	334
11.5.3 Presentaciones	334

Capítulo 12

Implementando el modelo en MySQL 5.1

12.1 Introducción	336
--------------------------------	------------

12.2 Inicializando el servidor	336	12.4.1 Creación de una conexión al servidor	344
12.2.1 Download	336	12.4.2 Creación de la base de datos, las tablas	
12.2.2 Instalación	337	y sus relaciones	345
12.2.3 Inicialización	341	12.5 Contenido de la página Web de apoyo	358
12.3 Inicializando el cliente	342	12.5.1 Mapa conceptual del capítulo	358
12.3.1 Download	342	12.5.2 Autoevaluación	358
12.3.2 Instalación	342	12.5.3 Presentaciones	358
12.3.3 Utilizando el MySQL Workbench	343	Bibliografía	359
12.4 Creando y probando nuestro modelo		Índice analítico	361
de Estudiante–Universidad	343		

Información del contenido de la página Web

El material marcado con asterisco (*) solo está disponible para docentes.

Capítulo 1.

Estructura y tipos de bases de datos

- Mapa conceptual
- Autoevaluación
- Presentaciones*
- Vínculos de interés

Capítulo 2.

Modelo de datos relacional

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 3.

SQL

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 4.

Lenguaje procedimental como extensión de SQL

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 5.

Bases de datos multidimensionales y tecnologías OLAP

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 6.

Almacén de datos

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 7.

Minería de datos

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 8.

Bases de datos orientadas a objetos

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 9.

Implementando el modelo en Oracle Express Edition XE

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 10.

Implementando el modelo en IBM DB2

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 11.

Implementando el modelo en SQL Server 2005

- Mapa conceptual
- Autoevaluación
- Presentaciones*

Capítulo 12.

Implementando el modelo en MySQL 5.1

- Mapa conceptual
- Autoevaluación
- Presentaciones*

- Lecturas complementarias
- Código fuente de los ejemplos
- Vínculos de interés

Incluye hipervínculos a la descarga de las versiones gratuitas de MySQL 5.1, SQL Server 2005, Oracle Express Edition XE e IBM DB2.

Registro en la Web de apoyo

Para tener acceso al material de la página Web de apoyo del libro:

1. Ir a la página <http://virtual.alfaomega.com.mx>
2. Registrarse como usuario del sitio y propietario del libro.
3. Ingresar al apartado de inscripción de libros y registrar la siguiente clave de acceso

4. Para navegar en la plataforma virtual de recursos del libro, usar los nombres de Usuario y Contraseña definidos en el punto número dos. El acceso a estos recursos es limitado. Si quiere un número extra de accesos envíe un correo electrónico a webmaster@alfaomega.com.mx

Estimado profesor: Si desea acceder a los contenidos exclusivos para docentes por favor contacte al representante de la editorial que lo suele visitar o envíenos un correo electrónico a webmaster@alfaomega.com.mx

Convenciones utilizadas en el texto

	Conceptos para recordar: bajo este icono se encuentran definiciones importantes que refuerzan lo explicado en la página.
	Comentarios o información extra: este ícono ayuda a comprender mejor o ampliar el texto principal.
	Contenidos interactivos: indica la presencia de contenidos extra en la Web.

Prólogo

En los últimos 20 años, el manejo de Base de Datos y el conocimiento de los Motores de Base de Datos existentes en el mercado han tomado un protagonismo absoluto en el desarrollo de software. Si bien este tema siempre se analizó dentro del área informática, antes solo era considerado para el desarrollo de sistemas de mediana y gran envergadura, pero actualmente no se justifica la construcción de una página Web sin tener como respaldo una base de datos para manejar la persistencia de los datos, aunque sean mínimos.

Por esta razón, el objetivo de este libro es introducir al lector en este tema mediante el concepto de composición de un Motor de Base de Datos, la forma de interactuar con él a través de los lenguajes asociados y los diferentes modelos de construcción lógica de datos.

En el Capítulo 1, se desarrollan los conceptos generales de las bases de datos y los Sistemas de Gestión de las mismas. Además, analiza conceptos como la seguridad de las mismas, la administración de usuarios y las funciones y las responsabilidades de un DBA (por sus siglas en inglés, *Data Base Administrator*).

En el Capítulo 2, se especifican los conceptos fundamentales del modelo relacional, trabajando el concepto de relaciones entre tablas, claves, integración, etcétera.

En el Capítulo 3, se especifican los comandos y la utilización del lenguaje de Consulta Estructurado (SQL), de forma tal que el lector conozca la forma de interactuar con un Sistema de Gestión de Base de Datos.

El Capítulo 4 extiende lo tratado en el capítulo anterior, pero también agrega el concepto de los lenguajes procedimentales considerando una extensión de SQL (PL/SQL) para poder agregar objetos creados por el diseñador a una base de datos específica.

En el Capítulo 5, se especifican las diferencias implementadas por las bases de datos multidimensionales y las distintas variantes de modelado aplicadas por las tecnologías OLAP.

Los Capítulos 6 y 7 establecen una aproximación a dos temas vinculados con las bases de datos como son el almacenamiento y la minería de datos. Si bien el objetivo del libro no es profundizar estos conceptos, se brinda una visión general de los mismos para conocimiento del lector.

En el Capítulo 8, se desarrolla el concepto de objetos aplicado a la persistencia de los datos administradas por un Sistema de Gestión. Además, se tratan las extensiones de bases de datos consideradas orientadas a objetos y las Bases de datos objeto-relacionales.

Por último, para poder trabajar sobre la práctica real, se desarrolló un modelo de datos que se implementa en Oracle Express Edition XE, IBM DB2, SQL Server 2005 y MySQL 5.1, en los Capítulos 9, 10, 11 y 12 respectivamente.

1

Estructura y tipos de bases de datos

Contenido

1.1 Introducción	2
1.2 Definición de base de datos	3
1.3 Sistema de Gestión de Bases de Datos	4
1.4 Usuarios de la base de datos	5
1.5 Seguridad de las bases de datos	7
1.6 Funciones y responsabilidades de un DBA	12
1.7 Arquitectura ANSI/SPARC	13
1.8 Modelos de datos	17
1.9 Resumen	23
1.10 Contenido de la página Web de apoyo	23

Objetivos

- Conocer la teoría de bases de datos.
- Identificar los modelos de datos anteriores y actuales para el almacenamiento persistente de grandes volúmenes de datos.
- Distinguir los tipos de usuarios y las funciones de un administrador de bases de datos.

1.1 Introducción



Entre 1960 y 1970, las computadoras tenían capacidades limitadas de procesamiento y almacenamiento.



Los datos que era necesario almacenar se ubicaban en archivos por orden físico.

Entre 1960 y 1970, las computadoras tenían capacidades limitadas de procesamiento y de almacenamiento. Si bien las aplicaciones no eran tan exigentes como las actuales, había que trabajar demasiado para que se lograran los resultados esperados.

Los datos, que era necesario almacenar, se ubicaban en archivos por un orden físico, cuya determinación se efectuaba de acuerdo con su orden de ingreso o por otro orden previamente indicado. Esto generaba mucho trabajo, ya que algún símbolo separaba los campos que conformaban los registros que, luego, se debían detectar para saber dónde finalizaba uno y empezaba el otro.

La lectura secuencial de los archivos era muy común y se debían recorrer todos los registros para encontrar el deseado; esta situación acarrearba problemas en la velocidad de respuesta.

Muchas veces, los distintos sectores de una empresa contaban con sus propios datos para resolver los problemas de información que se presentaban. Como no había ni políticas ni reglas claras, tampoco había personal informático que centralizara este tipo de situaciones, entonces, se repetían los datos.

Por esta razón, se necesitaba un esfuerzo mayor que el actual en la programación de aplicaciones para acceder a los datos, puesto que el almacenamiento se centraba en pocos tipos de datos, por ejemplo, numéricos y de texto. Esto limitaba las capacidades de respuesta a las necesidades de las empresas y exigía algoritmos de programación para un sinnúmero de situaciones, como el control y el almacenamiento de una determinada fecha.

En la actualidad, todas las personas interactúan con datos que, de alguna manera, se almacenan en un medio físico y se asocian a un sistema informático que los registra y permite su acceso. Por ejemplo: si desde su casa o desde un teléfono móvil, una persona marca un número de teléfono, lo que se hace es utilizar los datos registrados dentro de una empresa telefónica, que reconoce el teléfono, el origen de la llamada y el número del receptor. Eso implica una serie de acciones que desencadenan controles y registros en una base de datos. Con respecto al control, se deduce que éste debe verificar la existencia del número receptor o que, por ejemplo, no se encuentre ocupado; de lo contrario, es una llamada equivocada. Una vez establecida la comunicación, el sistema informático registrará mínimamente su número, el número destino, la hora de inicio y final de la llamada, al momento de dar por finalizada la comunicación. Dichos registros son los que permiten obtener la facturación de los clientes y de todos los reportes que se soliciten.

Lo mismo sucede cuando el cliente de un banco interactúa con el cajero automático para retirar dinero: el cliente debe ingresar su tarjeta, que se leerá para, luego, cruzar la identificación de la tarjeta con la clave que el usuario ingresa. La base de datos mantiene los datos del cliente y sus cuentas asociadas, mientras que una aplicación informática hará controles y mostrará las opciones disponibles al usuario.

En alguna sección de un sistema de gestión de alumnos de una universidad, el docente publica el programa de estudios de su materia, las fechas de los exámenes parciales y las notas de los estudiantes de su curso. También, publica los mensajes y los archivos para sus alumnos. El personal de la universidad que carga los resultados de

los exámenes indica las notas de los alumnos presentes y registra los ausentes. Otros usuarios acceden al sistema para cargar asistencias e inasistencias a clase de los alumnos. Por su parte, los alumnos ingresan con su legajo y su clave a la autogestión que presenta el sistema —por intranet o por Internet—, para ver la información de su estado académico, conformado por las materias que cursaron y las que cursarán, los resultados de sus exámenes —parciales y finales—, la cantidad de faltas que poseen e, incluso, se pueden inscribir en los exámenes de las materias que ya regularizaron.

Entonces, ¿qué es una base de datos? Algunos podrían pensar que son solo datos almacenados en una computadora como, por ejemplo, una planilla de cálculos. Sin embargo, es mucho más que eso.

1.2 Definición de base de datos

Cuando una empresa decide la implementación de un sistema informático, para cubrir necesidades de información específicas, el equipo de análisis y diseño inicia el proceso de desarrollo del software, basándose en las especificaciones con las que trabaja el cliente que solicita el sistema para, de esta manera, responder a las necesidades de la empresa.

Durante este proceso, se definen los archivos, como el de “Estudiantes”, para almacenar los datos de todos los alumnos de la universidad; el de los “Profesores”, para almacenar los datos de contacto necesarios; el de las “Carreras” que propone la universidad; y el de las materias que conforman cada plan de estudios vigente, o no, de cada carrera, etcétera.

Pero esos archivos nunca quedan aislados de otros datos; es decir: mantienen relaciones que, normalmente, son transparentes para los usuarios y que permiten su aprovechamiento para obtener información, a través de aplicaciones programadas para tales fines. Por ejemplo, es necesario tener algún archivo que mantenga los datos de la inscripción de cada alumno —en las carreras y en las materias que se cursarán—: esto permitirá que se reflejen los datos de resultados de los exámenes parciales, las asistencias, etcétera.

Por lo que hasta aquí se expresó, se puede ensayar una primera definición: “una base de datos es un conjunto de datos estructurados y definidos a través de un proceso específico, que busca evitar la redundancia, y que se almacenará en algún medio de almacenamiento masivo, como un disco”.

Cuando se hace referencia al término ‘redundancia’, se alude a la repetición que puede producirse en el momento de definir los almacenamientos de datos. Es incorrecta la pretensión de que los datos —como el apellido y la dirección— se encuentren definidos, a la vez, en los almacenamientos de estudiantes y en los relacionados con los exámenes finales. Un cambio en la dirección de un alumno podría significar que en uno de los dos almacenamientos no se realice la modificación y que provoque inconvenientes a la hora de devolver la información de un alumno.



Una base de datos es un conjunto de datos estructurados y definidos a través de un proceso específico, que busca evitar la redundancia, y que se almacenará en algún medio de almacenamiento masivo, como un disco.

1.3 Sistema de Gestión de Bases de Datos



Al SGBD se lo puede pensar como una capa de software que controla todos los accesos a la base de datos.

Cuando el alumno desea información de las notas que obtuvo durante el último año cursado, y que está almacenada en una base de datos, no accede directamente a ellos. Cada alumno de la universidad utiliza aplicaciones que ya han sido desarrolladas para fines específicos, como, por ejemplo, la obtención de un reporte de notas.

Por su parte, las aplicaciones que usa interactúan con un conjunto de programas aglutinados en lo que se denomina “el Sistema de Gestión de Bases de Datos (SGBD)”, “*Database Management System*” (DBMS) e, incluso, “Motor de Base de Datos”. A este Motor de Base de Datos, se lo puede pensar —de manera simplificada— como una capa de software que controla todos los accesos a la base de datos. Cabe aclarar, que un DBMS, no se crea para una situación específica de una empresa, sino que se desempeñará tanto para fines de un sistema de gestión de alumnos como para un sistema bancario, una empresa telefónica, comercial, etcétera.

El DBMS puede implementar instrucciones dadas por los distintos usuarios, que se describen en las próximas páginas, y que tienen distintos efectos en una base de datos. Las instrucciones se agrupan mínimamente en: DDL (Lenguaje de Definición de Datos) y DML (Lenguaje de Manipulación de Datos), aunque también suele reconocerse al DCL (Lenguaje de Control de Datos). A manera introductoria, se indican algunas instrucciones que conforman estos grupos:

DDL: es el conjunto de órdenes que permiten definir la estructura de una base de datos. Por ejemplo, para crear la base de datos en una empresa se utiliza una orden de este tipo, acompañada de los parámetros necesarios e indicados en cada DBMS. Lo mismo sucede si se desea modificar la estructura de un objeto de la base de datos, como por ejemplo cuando en un archivo de CLIENTES se desea agregar un dato nuevo no considerado al ser creada la estructura. Además incluye una instrucción para eliminar objetos obsoletos en la base de datos. No son instrucciones a incluir en las aplicaciones.

DML: las instrucciones que conforman este grupo son las que están incluidas en las aplicaciones y se usan para alterar el contenido de un archivo de datos. Por ejemplo, cuando se desea insertar un nuevo cliente o producto, modificar la dirección de un cliente o el precio de un producto, o eliminar un producto que fue cargado por error. Nótese la diferencia respecto al DDL que actuaba sobre la estructura para almacenar, mientras que estas órdenes afectan al contenido de los archivos de datos.

DCL: son órdenes que se utilizan para implementar seguridad en la base de datos, como por ejemplo indicar que privilegios –insertar, modificar, etc.– tiene cada usuario respecto a los distintos objetos de la base de datos. Incluso pueden ser retirados privilegios a usuarios existentes.

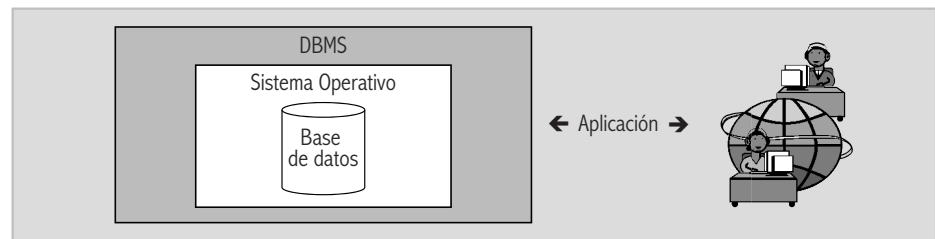


Fig. 1.1 Representación gráfica de una base de datos.

Si bien las aplicaciones poseen controles de seguridad, al solicitar un nombre de usuario y contraseña cuando el usuario ingresa al sistema, siempre hay una segunda instancia de seguridad definida por personal que implementa la base de datos, donde se definen al DBMS perfiles de usuarios y privilegios que posee respecto a los datos, por ejemplo si un usuario Carlos puede o no agregar nuevos datos, o modificar datos almacenados en la base de datos.

Teniendo en cuenta lo expresado, ahora podemos pensar en una definición que amplíe el concepto de bases de datos enunciado en el punto 1.2. Para ello, se utilizará una cita —muy utilizada por otros autores— que abarca las nociones hasta aquí enunciadas. James Martin en su obra de 1975 *Computer Data-base Organization* define base de datos como:

“Colección de datos interrelacionados almacenados en conjunto sin redundancias perjudiciales o innecesarias; su finalidad es servir a una aplicación o más, de la mejor manera posible; los datos se almacenan de modo que resulten independientes de los programas que los usan; se emplean métodos bien determinados para incluir nuevos datos y para modificar o extraer los datos almacenados”.

Esta definición incluye el concepto de independencia de datos, que significa que las aplicaciones deben ser independientes de las modificaciones que pudieran sufrir los datos, en lo lógico y en lo físico. Sería deseable que esto se lograra en las bases de datos, aunque, en algunas situaciones, resulte más o menos complejo.

Es decir que si, por ejemplo, en el almacenamiento “Estudiantes” de la base de datos se agrega un dato —como la cuenta de correo electrónico o el número de teléfono móvil—, no implica que en los programas que usan el almacenamiento se deba hacer cambios. Eso es lo que se denomina **independencia lógica**. En otras épocas, las aplicaciones incluían la definición de la estructura de datos y había una sección en la que figuraban todos los datos que poseía cada uno de los archivos de datos, a los que se accedía por dicho programa. Esto significaba que, ante un cambio en la estructura, el agregado de un nuevo dato o la alteración de la dimensión de un dato existente, las aplicaciones —que podían ser muchas— debían reflejar el cambio en la sección correspondiente.

En cuanto a la **independencia física**, el almacenamiento “Estudiantes” puede definirse por la indexación, que es el acceso rápido por el legajo del estudiante. Si esta forma de acceso se cambiara para mejorar la *performance* del sistema, no debería determinar un cambio en las aplicaciones y, si se decidiera la utilización de otro almacenamiento físico para la base de datos, tampoco se deberían generar cambios en las mencionadas aplicaciones.

1.4 Usuarios de la base de datos

Hay diversas clasificaciones de los usuarios que actúan en un entorno de bases de datos, como las planteadas por grandes autores como C.J. Date en *Sistemas de Bases de Datos*, Korth en *Fundamentos de Bases de Datos* y Ramez Elmasri y Shamkant Navathe en *Sistemas de Bases de Datos: conceptos fundamentales*.

Se estudiarán distintos momentos de un sistema con bases de datos y se irán detectando los perfiles que, actualmente, se repiten en las empresas, a los que les asignaremos una denominación y, de esta manera, quedarán clasificados.

Cuando se desea informatizar un nuevo sistema de información, es imprescindible que participe un equipo de análisis y diseño de sistemas, conformado por ingenieros



Independencia de datos significa que las aplicaciones deben ser independientes de las modificaciones que pudieran sufrir los datos, en lo lógico y en lo físico.



Christopher J. Date. Investigador principal del modelo relacional de bases de datos de Edgar Codd.



En la Web de apoyo, encontrará el vínculo a la página Web personal de Shamkant B. Navathe.

y analistas de sistemas. Este equipo es el que toma los requerimientos del cliente mediante entrevistas, cuestionarios, etc., y plasma —en diagramas y descripciones— la interpretación de ese relevamiento. Luego entrará en un intenso proceso ingenieril, que incluirá el análisis, la creación y el diseño. El equipo planteará una propuesta al cliente —con los alcances y los límites bien determinados— para que la revise y mejore. Si se acepta la propuesta, se mejorará el nivel de detalle del diseño, se intercambiarán opiniones con los usuarios futuros con respecto a las interfaces y los reportes del sistema, para estipular cuáles serán las entidades que participarán en el desarrollo de las aplicaciones, cuáles serán las estructuras de cada una, quiénes serán los usuarios y sus respectivos roles, etcétera.



Usuarios de la base de datos:

1. Administrador de la base de datos.
2. Programador de aplicaciones.
3. Usuarios final.

De esta manera, se resume un proceso muy importante, que llevará días o meses de trabajo, según la complejidad del sistema; pero la intención es indicar que este equipo no interactúa necesariamente con el sistema en producción, sino que lo hace en la etapa previa a la construcción. Por esta razón, no se lo considera usuario del sistema.

Después de reconocer las entidades y relaciones entre ellas, se pueden detectar usuarios que interactúan con la base de datos de distinta manera y con distintos objetivos. La clasificación que se propone a continuación no es la única que suele expresarse, muchos autores indican otras con diferentes roles en la relación del usuario con la base de datos.

- Administrador de la base de datos: comúnmente es el profesional—ingeniero o analista— con perfil técnico que, en el ambiente informático, se denomina DBA (*Data Base Administrator*). Este profesional, muy importante para la empresa, recibe las especificaciones del Equipo de Análisis y Diseño para su implementación en un Sistema de Gestión de Base de Datos, como por ejemplo, Oracle, SQL Server, DB2, etcétera. El DBA tiene múltiples funciones y responsabilidades que se detallarán más adelante.
- Programador de aplicaciones: conoce los casos que se desarrollarán —escritos e identificados por el Equipo de Análisis y Diseño—, los prototipos de interfaces y las estructuras de los almacenamientos que se manipularán. El programador genera las aplicaciones necesarias en el sistema —con el lenguaje de programación que se le indica y conoce—, para la obtención de las entradas de datos que alimentarán la base de datos y, también, para lograr las salidas —como las pantallas de resultados o reportes—, que se plantearon en la propuesta de solución. Es conveniente aclarar que no hace falta que este usuario conozca toda la estructura de la base de datos, sino solo lo que necesita para programar. Normalmente, el programador de aplicaciones trabaja en un equipo de desarrollo, cuyo tamaño dependerá de la envergadura del sistema.
- Usuario final: es el personal que interactúa con las aplicaciones programadas por el usuario mencionado en el párrafo precedente y es, de entre todos los usuarios, el que menos conocimiento técnico posee. Si bien hay perfiles distintos entre los usuarios finales que interactúan con el sistema, esta persona no conoce ni los detalles técnicos ni los de la base de datos. Por ejemplo: un alumno que accede a su autogestión vía Internet para conocer su estado académico no necesita conocer la estructura del sistema, solo utiliza las opciones que se le presentan en el menú, que están asociadas con un perfil definido

para todos los alumnos. Otro usuario final del Sistema de Gestión de Alumnos es el profesor de un curso, quien verá las opciones definidas para ese perfil, con la posibilidad de cargar las notas de los parciales, enviar mensajes a todo el curso o imprimir la lista de sus alumnos.

Durante la vida del sistema, seguramente, se presentarán requerimientos que cambiarán y otros que no fueron considerados cuando se diseñó el sistema. En este último caso, es lógico que determinados usuarios no tengan los programas que necesitan. Por ejemplo, es muy común que el personal directivo especifique una necesidad como, por ejemplo, “Obtener los datos de los alumnos que cursan el último año de la carrera, trabajan, tienen promedio superior a 9 (nueve) y que no han rendido materias en los últimos dos turnos de examen”. Este tipo de solicitud no programada suele dar lugar a que ciertos usuarios tengan permisos para acceder a la base de datos mediante consultas que ellos mismos arman en pantalla, en las que indican los almacenamientos que usarán, las condiciones que cumplirán, los datos que mostrarán y los cálculos que realizarán. Si esta solicitud se repitiera en el tiempo, se la podría incluir como una opción del sistema.

1.5 Seguridad de las bases de datos

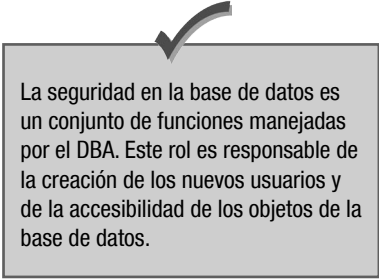
La seguridad en la base de datos es un tema muy importante. En la mayoría de las organizaciones, es un conjunto de funciones manejadas por el Administrador de la Base de datos (o DBA) o, más apropiadamente, por un administrador de seguridad. Este rol es responsable de la creación de los nuevos usuarios y de la accesibilidad de los objetos de la base de datos. Como regla general, cuanto más grande es la organización, más sensible es la información y más probable es que la seguridad sea manejada por un Administrador de seguridad. Sin embargo, es importante que los desarrolladores, los DBA y los usuarios entiendan las opciones disponibles en el modelo de seguridad.

La base de datos más segura es aquella que no tiene usuarios, pero esta situación carece de sentido. Por esta razón, se debe llegar a un balance entre el permiso de acceso a los usuarios y el control de lo que se les permite hacer cuando establecen una sesión a través de una conexión.

Para ello, existe en cada base de datos un modelo de seguridad. Por razones de síntesis, nos dedicaremos a estudiar el que ofrece la base de datos Oracle. Se recomienda visitar la página de documentación oficial de la empresa en:
<http://www.oracle.com/technetwork/database/focus-areas/security/index.html>

En Oracle, el modelo de seguridad se basa en los siguientes elementos:

- Privilegios de sistema.
- Uso de roles para administrar el acceso a los datos.
- Privilegios de objeto.
- Contraseñas modificables.
- Otorgar y revocar los privilegios de objetos y de sistema.
- Uso de sinónimos para la transparencia de base de datos.



La seguridad en la base de datos es un conjunto de funciones manejadas por el DBA. Este rol es responsable de la creación de los nuevos usuarios y de la accesibilidad de los objetos de la base de datos.

El modelo de seguridad Oracle consiste en dos partes:

1. Autenticación de contraseñas, que puede basarse en el Sistema Operativo que aloja al motor o usar el propio sistema de autenticación de la base. Cuando la última opción es la elegida por el DBA, la información de contraseñas se almacena en un formato encriptado en el diccionario de la base de datos.
2. Control de qué objeto de la base de datos se puede acceder y por cuál grupo de usuarios. Esta autoridad se transmite a través de los privilegios que se le asignan a los usuarios directamente o por medio de roles a grupos de usuarios.

1.5.1 Creación de usuarios

El inicio del proceso —que es el que permite el acceso a la base de datos— es la creación de usuarios con el comando SQL:

CREATE USER <nombre> **IDENTIFIED BY** <contraseña>

Usualmente se completa con algunas cláusulas de almacenamiento y opciones de usos de la base de datos, pero que no se relacionan con el tema que trataremos aquí.

1.5.2 Otorgar privilegios de sistema

Los privilegios de sistema permiten al usuario que los recibe la creación, modificación y eliminación de los objetos de la base de datos que almacenan los datos de las aplicaciones.

De hecho, para poder realizar alguna operación en la base de datos, el usuario necesita un privilegio de sistema para ingresar, denominado *CREATE SESSION*.

Dentro del alcance de los privilegios de sistema hay dos categorías:

- La primera de ellas es el conjunto de privilegios de sistema que se relacionan con el manejo de los objetos como tablas, índices, disparadores o *triggers*, secuencias, vistas, paquete, procedimientos y funciones almacenadas. Con estos privilegios se pueden realizar las siguientes acciones: crear, alterar su definición y eliminarlos en general y, para las tablas y programas, tenemos los privilegios de creación de índices, hacer referencia a la clave primaria con una clave foránea y ejecutar un programa almacenado.
- La segunda categoría de los privilegios se refiere a la habilidad de un usuario para realizar actividades especiales a lo largo del sistema. Estas actividades incluyen funciones como: actividades de auditoría, generación de estadísticas para soportar el funcionamiento del optimizador basado en costos y permitir el acceso a la base de datos solamente a los usuarios con un privilegio de sistemas especial denominado “sesión restringida” o *restricted session*. Estos privilegios usualmente se deberían otorgar solo a los usuarios que realizarán tareas de administración de alto nivel.

El otorgamiento de los privilegios de sistemas se ejecuta con la sentencia *GRANT* y, para otorgar un privilegio de sistema a otro usuario, el que lo transmite (*GRANTOR*) debe haber recibido el mismo privilegio con el agregado *WITH ADMIN OPTION*, para poder retransmitirlo a otros usuarios.



Los privilegios de sistema permiten al usuario que los recibe la creación, modificación y eliminación de los objetos de las bases de datos que almacenan los datos de las aplicaciones.

Los usuarios pueden crear objetos dentro de su esquema y si fuera necesario que un usuario deba crear objetos para cualquier otro esquema, deberá recibir el privilegio con la palabra *ANY* como en el siguiente ejemplo:

GRANT CREATE ANY TABLE TO miusuario

La anulación de un privilegio otorgado se realiza con la sentencia *REVOKE*. Si un usuario recibió un privilegio de sistema y lo ha transmitido con la opción *WITH ADMIN OPTION*, el hecho de perderlo no afectará a los usuarios que lo recibieron.

Por ejemplo: el usuario *PEPE* recibió el privilegio *CREATE ANY TABLE WITH ADMIN OPTION* realizó su trabajo creando cinco tablas y le pasó el mismo privilegio a dos usuarios más, a uno con la opción *WITH ADMIN* y a otro sin ella. Si otro usuario le revoca este privilegio al usuario *PEPE*, esta acción no tiene efecto sobre las tablas ni sobre los dos usuarios, ya que no pierden su privilegio de creación de tablas en cualquier esquema.

1.5.3 Usar roles para administrar los accesos de base de datos

Cuando la cantidad de objetos y usuarios va creciendo, la administración de accesos y privilegios se torna complicada y trabajosa, porque se van agregando aplicaciones que traen entre cientos y miles de objetos y se incorporan nuevos usuarios o los que están adquieren nuevos privilegios de acceso sobre estos objetos en forma acorde con la operativa.

La forma de simplificar elegida por los Sistemas de Gestión de Base de Datos ha sido la creación de una capa de abstracción llamada *ROL*, que agrupa los privilegios y, generalmente, define un nombre de rol que describe un tipo de tarea o función. Además de simbolizar un tipo de tarea o actividad general, con el *ROL* se establece la idealización de un usuario que necesita un grupo de privilegios para un cierto trabajo, como *ESTUDIANTE*, *PROFESOR*, *VENDEDOR*, *OPERADOR*, *DATAENTRY*, *SUPERVISOR*, *OPERADOR_BATCH*, *OPERADOR_BACKUP*. Así, todos los accesos a los objetos se pueden agrupar en estos roles de base de datos para ser administrados y otorgados entre los usuarios y entre otros *ROLES*, ya que es posible asignar *ROLES* a otros *ROLES*.

Para usar *ROLES* es necesario realizar dos tareas:

1. Agrupar lógicamente los privilegios, como los de creación de tablas, índices y procedimientos. También se puede definir una contraseña para un *ROL* para, de esta manera, proteger el acceso a los privilegios con la cláusula idéntica a la creación de usuarios. Veamos el ejemplo:

CREATE ROLE creador_procs IDENTIFIED BY soycreador

GRANT create any procedure TO creador_procs WITH ADMIN OPTION

2. La segunda tarea que se debe completar es agrupar a los usuarios de cada aplicación que manejan necesidades similares. El modo más efectivo es la identificación de los distintos tipos de usuarios que usarán la base de datos. Se determinan las actividades que cada tipo de usuarios realizará y se listan los privilegios que serán necesarios para llevar a cabo esta tarea. Luego, se creará un *ROL* por cada actividad y se otorgará con *GRANT* los privilegios al *ROL* definido.



La forma de simplificar elegida por los Sistemas de Gestión de Bases de Datos ha sido la creación de una capa de abstracción llamada *ROL*.



Para usar *ROLES* es necesario realizar dos tareas:

1. Agrupar lógicamente los privilegios.
2. Agrupar a los usuarios de cada aplicación que manejan necesidades similares.

Esta forma de hacer niveles de ROLES como capas intermedias, para distribuir privilegios establecidos, facilita la administración, puesto que bastará con determinar el tipo de usuario que corresponde o la clase de tarea que se le habilitará para encontrar uno o dos ROLES. De esta manera, el usuario recibirá todos los privilegios necesarios.

En el ejemplo, ilustramos esta secuencia de tareas:

```
CREATE ROLE desarrollador
GRANT CREATE TABLE TO desarrollador
GRANT SELECT ANY TABLE TO desarrollador
GRANT DROP USER TO desarrollador
GRANT desarrollador TO juan
GRANT desarrollador TO pedro
```

Los roles se pueden modificar para que soporten el requerimiento de contraseña usando la siguiente sentencias:

```
ALTER ROLE desarrollador IDENTIFIED BY abrepuertas
```

Los roles se eliminan con la ejecución de la sentencia *DROP ROLE*. Esta acción requiere que el administrador tenga los siguientes privilegios: *CREATE ANY ROLE*, *ALTER ANY ROLE* y *DROP ANY ROLE*, o que sea dueño del ROL.

Los roles se pueden otorgar a otros roles con *GRANT* y se quitan de los usuarios o roles con *REVOKE*.

El hecho de que un ROL tenga contraseña, nos hace preguntar en qué momento y cómo se activa para ejercer los privilegios asociados. Al inicio de la sesión, se activan los roles fijados como *default* para el usuario. Normalmente, el estado de un ROL es *ENABLED* o activado, a menos que haya sido expresamente desactivado. Estas acciones se realizan con las siguientes sentencias:

```
ALTER USER juan DEFAULT ROLE ALL (Activa todos los roles asignados a juan)
ALTER USER juan DEFAULT ROLE ALL EXCEPT sysdba
ALTER USER juan DEFAULT ROLE desarrollador, operador_backup
ALTER USER juan DEFAULT ROLE NONE (ningún ROL queda activado al iniciar la sesión)
```

1.5.4 Otorgar privilegios de objetos

Una vez que se ha creado, un objeto puede ser administrado por su creador o por el usuario que tenga el privilegio *GRANT ANY PRIVILEGE*.

La administración consiste en otorgar los privilegios sobre el objeto que permitirán manipularlo: agregando filas, cambiando su estructura o viendo los datos de sus columnas.

Los privilegios de objetos se agrupan en dos grupos: los que permiten ver, agregar, modificar e insertar filas y los de objetos que dejan eliminar otros objetos o usarlos como el privilegio *REFERENCE* y *EXECUTE*, que permiten crear claves foráneas en una tabla que referencia a una clave primaria de otra tabla sobre la que se ha dado este privilegio de referencia. El privilegio *EXECUTE* otorga a un usuario la facultad de



Los privilegios de objetos se agrupan en dos grupos: los que permiten ver, agregar, modificar e insertar filas y los de objetos que dejan eliminar otros objetos o usarlos como el privilegio *REFERENCE* y *EXECUTE*.

ejecutar una función, procedimiento o paquete almacenado. Otros privilegios administran la modificación y creación de objetos de base de datos, como *ALTER TABLE*, *ALTER SEQUENCE* y el privilegio que permite crear un índice sobre una tabla de otro usuario es *INDEX TABLE*.

Estos privilegios de objetos pertenecen al usuario que los creó o que los posea con *ANY*. Se otorgan también con una opción que facilita su transmisión a otros usuarios, como es el parámetro *WITH GRANT OPTION*.

A diferencia de los privilegios de sistemas, un privilegio de objeto otorgado por un usuario que lo recibió con el parámetro *WITH GRANT OPTION* se revocará automáticamente si a este usuario *GRANTOR* se le anulara este privilegio. Es decir: hay un efecto cascada en el *REVOKE* para los privilegios de objeto que no ocurre con los privilegios de sistema.

1.5.5 El cambio de contraseñas

Una vez creado, el usuario puede cambiar su contraseña con la sentencia:

```
ALTER USER juan IDENTIFIED BY abretesesamo
```

1.5.6 Uso de sinónimos para la transparencia de base de datos

Los objetos de base de datos son de los usuarios que los crean y solo estarán disponibles para él, a menos que se otorguen explícitamente privilegios a otros usuarios o roles.

Sin embargo, y pese a que otro usuario tenga privilegios para usar estos objetos de otro esquema, al hacer uso de ellos debe respetar los límites del esquema; esto es: debe llamar al objeto con su denominación completa en la base, es decir, el nombre del esquema precederá al nombre del objeto. Por ejemplo: si Juan desea usar la tabla clientes del esquema Pedro, usará el nombre del esquema como prefijo del nombre de la tabla. Si Juan ya tiene permiso sobre la tabla "Materias" de Pedro, incluirá el nombre del esquema de esta manera:

```
SELECT * FROM pedro.materias
```

Sin la información del nombre del esquema al que pertenece el objeto, el motor de la base de datos no entenderá de qué tabla Materias se trata.

Este requisito de anteponer el esquema produce la pérdida de transparencia que debe resolverse usando "sinónimos", ya que permite a Juan ver la tabla Clientes sin necesidad de anteponer explícitamente el nombre del esquema.

Hay dos tipos de sinónimos:

- Privados: tienen una aplicación más limitada.
- Públicos: se usan para poner a disposición de los objetos a todos los usuarios sin el requisito de calificarlos con el formato "esquema.nombre_del_objeto".

Si se realiza la siguiente operación:

```
CREATE PUBLIC SYNONYM materias FOR pedro. materias
```



El requisito de anteponer el esquema produce la pérdida de transparencia que debe resolverse usando "sinónimos".

Ahora Juan (y cualquier otro usuario) podrá referirse a la tabla Materias de Pedro solamente con el sinónimo del objeto, sin prefijo del esquema.

*SELECT * FROM materias*

Los privilegios privados se crean con el mismo fin, pero tienen alcance limitado para cada usuario en particular, como:

CREATE SYNONYM materias FOR pedro. materias

Ahora solamente Juan podrá hacer esto con la tabla de Pedro:

*SELECT * FROM materias*

Con estos componentes del modelo de seguridad de la base de datos, el usuario podrá entender la mayoría de los modelos de otras bases, ya que implementan esencialmente las mismas características, pero deben observarse las diferencias que pudieran existir en cada implementación.

1.6 Funciones y responsabilidades de un DBA



El DBA indica cuáles son los datos de cada entidad, los tipos de datos, la dimensión de cada dato, las relaciones entre ellos, sus claves identificadas, las vistas para los usuarios finales, etc.

Como se afirmó en los párrafos precedentes, el DBA tiene múltiples funciones técnicas y responsabilidades con el sistema. En algunas organizaciones, suele trabajar solo, pero es muy común que necesite personal a su cargo para realizar tareas operacionales y para cubrir la alta demanda de tiempo; por ejemplo, para atender los requerimientos de empresas con servicios internacionales y con uso del sistema en 7 días x 24 horas, es decir, todas las horas de todos los días de la semana del año, como en el caso de un sistema de vuelos internacionales.

Entre sus funciones principales se pueden mencionar:

- Especificación lógica de la base de datos: el DBA especifica —mediante la interfaz del DBMS elegido o las sentencias de definición de datos— la estructura de la base de datos que recibió del Equipo de Análisis y Diseño. Indica cuáles son los datos de cada entidad, los tipos de datos, la dimensión de cada dato, las relaciones entre ellos, sus claves identificadas, las vistas para los usuarios finales, etcétera. Las sentencias que utiliza el DBA se agrupan en lo que se denomina Lenguaje de Definición de Datos (DDL) y está formada por las instrucciones para trabajar en la estructura y, también, para crear objetos, para modificar las estructuras de los objetos o para eliminar los que ya no se necesiten en la base de datos.
- Especificación física: el DBA define el medio físico que almacenará a la base de datos; por ejemplo, el disco y su partición en el servidor y, también, cómo se almacenarán los archivos de datos y cómo se accederá a ellos para lograr una mejor *performance*.
- Definición de seguridad: define grupos de usuarios y usuarios individuales, con los perfiles para cada uno, e indica los archivos a los que pueden acceder y los derechos que poseen de manera individual. Por ejemplo: el perfil “Estudiante” permite el derecho de consultar los archivos que contienen los datos de sus

evaluaciones; de consultar y enviar mensajes a los docentes; de modificar su contraseña. Sin embargo, no tendrá la posibilidad de ver los datos personales de sus docentes o el derecho a modificar las notas de sus evaluaciones.

- Definir procedimiento de respaldo: es responsabilidad del DBA asegurar que los datos estén respaldados para evitar inconvenientes ante algún tipo de incidente (rotura de medio físico, errores de procedimientos en actualizaciones, robo de hardware, incendio, etc.). Por lo tanto, deberá definir la periodicidad de los respaldos, el o los medios para mantenerlos (back-up); si copiará todo el contenido de la base de datos cada vez que inicie el respaldo, o si será parcial, etcétera. Esto no significa que el DBA será el que realice este trabajo, sino que es el que define el procedimiento que ejecutará el personal de soporte en la empresa. Del mismo modo, designará a los encargados del procedimiento para la recuperación de datos.
- Implementar reglas de integridad: el Equipo de Análisis y Diseño especifica ciertas limitaciones a los datos que se almacenarán, determinados por el mundo real de la organización o por reglas lógicas propias de las bases de datos. Por ejemplo: en determinados países, la nota que un estudiante puede obtener está entre el valor 0 (cero) y el 10 (diez), lo que implica que el dato “Nota” nunca estará fuera de esos límites. Otro ejemplo de integridad se observa en un archivo de “exámenes”, en el cual al estudiante que se registra, debe ser alumno regular de la universidad y, la materia que rinda, parte integrante del plan de estudios de la carrera que cursa. Si se analiza un sistema bancario, también tiene las restricciones propias del negocio al igual que una empresa de telefonía. Estas reglas se definen durante el proceso de creación de las bases de datos y de los archivos que la conforman; incluso, se pueden agregar algunas nuevas que surjan durante el uso del sistema y mientras los datos lo permitan, ya que algunos, seguramente, existen.
- Monitorear la *performance* de la base de datos: el DBA debe monitorear el rendimiento de la base de datos, detectando los procesos que generen demoras en la devolución de información, para mejorar, permanentemente, su *performance* general. En algunos casos, el resultado es que los desarrolladores deben revisar las aplicaciones o la arquitectura del sistema; en otras situaciones, se pueden mejorar los métodos de acceso; incluso, la ubicación de los archivos de datos o las capacidades del hardware o de la conectividad que influyen en el rendimiento. El DBA resuelve determinadas situaciones pero, en otras, solo se informará a quienes tienen el poder de decisión, para que encuentren las soluciones posibles.

1.7 Arquitectura ANSI/SPARC

Como las funciones de los distintos usuarios mencionados de una base de datos son distintas, lo que implica que cada uno puede interactuar de diversas formas con los datos almacenados, surge la Arquitectura ANSI/SPARC para estandarizar los conceptos y permitir una mejor lectura de la independencia de datos, lo que permitirá que se ubique a cada usuario de una base de datos en función de su relación con ella, ya que no todos poseen la misma visión, aunque los datos almacenados son únicos.



ANSI es el instituto que "supervisa la creación, promulgación y el uso de miles de normas y directrices que impactan directamente en casi todos los sectores de las empresas: desde dispositivos acústicos hasta los equipos de construcción, desde la producción lechera y ganadera hasta la distribución de energía, y muchos más".

ANSI (*American National Standards Institute*) tuvo mucha influencia en la definición de esta arquitectura. Fundado el 19 de octubre de 1918, se autodefine, en su página Web (www.ansi.org), como el instituto que "supervisa la creación, promulgación y el uso de miles de normas y directrices que impactan directamente en casi todos los sectores de las empresas: desde dispositivos acústicos hasta los equipos de construcción, desde la producción lechera y ganadera hasta la distribución de energía, y muchos más".

La arquitectura, que se presentará en tres niveles, tiene su antecedente en 1971, cuando CODASYL (*Conference on Data Systems Languages*). Además, este comité, formado por personas de industrias informáticas, tiene el objetivo de regular el desarrollo de un lenguaje de programación estándar, de donde surge el lenguaje COBOL, reconoce la necesidad de una arquitectura en dos niveles: un esquema —vista de sistema— y un subesquema —vista de usuario.

Posteriormente, ANSI se dedica al tema, como lo dice en el libro *Databases, role and structure: an advanced course*, escrito por P. M. Stocker, P. M. D. Gray y M. P. Atkinson, en el cual indica que cuenta con un comité denominado X3, que se dedica a la computación y al procesamiento de información. El SPARC (*Standards Planning and Requirements Committee*) de ANSI/X3 fija un grupo de estudio de DBMS para especificar los modelos para la estandarización de las bases de datos. En 1972, este comité produce un reporte provisional, seguido por otro final, en 1977, el cual divide la vista de sistema —destacada por CODASYL— en vista de empresa y de almacenamiento, que es lo que actualmente se conoce como:

- **Nivel externo:** lo conforman las múltiples vistas de los datos almacenados en la base de datos y se presentan a los distintos usuarios de múltiples formas, adecuándolas a las necesidades de información que tiene cada uno. A este nivel también se lo denomina **nivel de visión**, que se define con el lenguaje de manipulación de datos.
- **Nivel conceptual:** es la estructura lógica global, que representa las estructuras de datos y sus relaciones. Hay una única vista en este nivel y se la define con el lenguaje de definición de datos.
- **Nivel interno:** se define como el conjunto de datos que están almacenados físicamente y, como no se accede a ellos, también se lo denomina **nivel físico**.

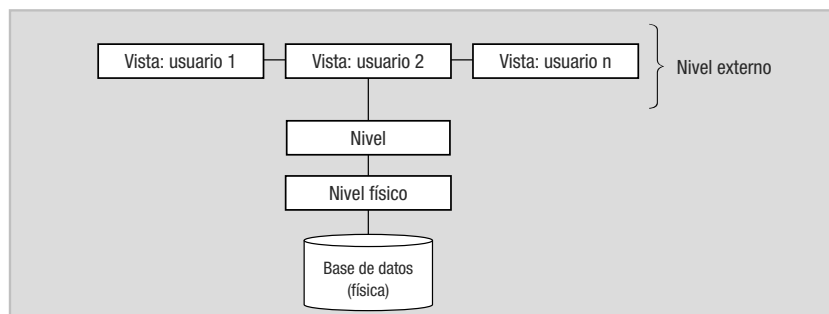


Fig. 1.2 Niveles de la base de datos.

A continuación, se verá la relación de los usuarios con los niveles de abstracción expresados en la Fig. 1.2. Los usuarios finales y los programadores de aplicaciones no

necesitan conocer cómo están almacenados los datos físicamente, porque tienen una visión alejada del nivel físico y personalizada para cubrir las necesidades de cada uno. Por ejemplo, un cajero de una entidad bancaria no precisa los mismos datos que un gerente, ni un estudiante universitario las opciones que usa un profesor de la universidad. Un reporte presentado a un gerente, puede ser el resultado de un largo proceso en el cual se combinaron los datos provenientes de distintos archivos y se aplicaron múltiples cálculos. Es lo que necesita, ya que no conoce ni le interesan estos detalles técnicos.

Otro usuario, el DBA, está en condiciones y es el único que debería conocer el nivel físico, para modificarlo si hiciera falta. Por ejemplo: podría cambiar el medio de almacenamiento de la base de datos o la forma de acceso a los archivos, manteniendo siempre inalterables las vistas para los demás usuarios.

Solo el DBA puede introducir cambios en el nivel conceptual, pero sí es necesario que mantenga las vistas inalteradas hacia los usuarios finales y los programadores de aplicaciones. Por ejemplo: agregar una columna a un archivo de datos cuando éste no se lo tuvo en cuenta inicialmente en el sistema o agregar un archivo de datos con las relaciones que corresponda a los ya existentes.

Un archivo de datos de “Productos” y otro de “Proveedores”, de una base de datos de una empresa que vende insumos y artículos informáticos, podrían tener este aspecto:

Ejemplo:

Tabla Productos					
CodProd	DesProd	PrecioVta	CodProv	Stock	StMin
108	IMP. MULTIF. TX410	699,80	19	2	6
230	MON. 19 PULG. LCD W1943C	925,50	32	12	10
110	IMP. MULTIF. C5580-CH.TIN.	769	21	3	6
...	...	...	...	...	...

Tabla Proveedores			
CodProv	RazónSocial	Contacto	Teléfono
21	HP Center	...	...
19	Moura EPSON	...	...
32	Asoc. LG	...	...
...	...	...	...



Los archivos de la base de datos se almacenan de una única manera que, en definitiva, es un conjunto de bytes que no son visibles para ningún tipo de usuario y que se administra a través del sistema operativo.

Físicamente, estos archivos de la base de datos se almacenan de una única manera que, en definitiva, es un conjunto de *bytes* que no son visibles para ningún tipo de usuario y que se administra a través del sistema operativo. Pero, el DBA, si utiliza

herramientas e instrucciones del DBMS, lo puede organizar e indicará en qué disco se almacena la base de datos (disco local o remoto, en uno u otro servidor) y definirá, para cada archivo, su método de acceso. Por ejemplo: mediante la generación de un índice por código de producto en el archivo “Productos” y, de esta manera, agilizará su búsqueda.

En el nivel conceptual o de estructura lógica, el DBA puede crear o modificar las estructuras de estos archivos. Por ejemplo: si indica que el dato “Código del Producto” es numérico, con cuatro dígitos posibles (0 a 9999), se autogenera cada vez que se inserta un nuevo producto, sin que pueda repetirse. Aquí, también, es necesario identificar la relación entre los archivos de “Productos” y de “Proveedores” y determinará cómo se produce una búsqueda o qué sucede cuando un proveedor quiere que se lo elimine de la base de datos.

En el nivel externo, cada usuario verá lo que necesita y tiene habilitado; por ejemplo:

- El vendedor verá, en su pantalla, un listado ordenado por descripción del producto, con cabeceras de listado distintas a los nombres de los campos de datos en el archivo origen. A él no le interesa ni quién es el proveedor ni cuál es el *stock* mínimo de cada producto. Tampoco sabe si todos estos datos provienen de un solo almacenamiento o de una combinación.

Producto	Descripción	Precio	Stock actual
108	IMP. MULTIF. TX410	699,80	2
110	IMP. MULTIF. C5580-CH.TIN.	769	3
230	MON. 19 PULG. LCD W1943C	925,50	12
...	...	...	...

- El encargado de compras visualizará un listado con combinación de datos de los dos archivos, con cabeceras más descriptivas, que incluirá el resultado de un cálculo e indicará lo que debería comprar, como mínimo, para cubrir el *stock* por producto:

Stock						
Producto	Descripción	Precio	Actual	Mínimo	Comprar	Proveedor
108	IMP. MULTIF. TX410	699,80	2	6	4	Moura EPSON
230	MON. 19 PULG. LCD W1943C	925,50	12	10	0	Asoc. LG
110	IMP. MULTIF. C5580-CH.TIN.	769	3	6	3	HP Center
...	...	...	...	...	...	...

- El gerente de la empresa, seguramente, necesitará un informe con una mayor complejidad de proceso, que incluirá los datos estadísticos resultantes de los cálculos que resumirá una situación, en la que se podría utilizar hasta un gráfico en pantalla; pero, en definitiva, los obtendrá de los datos almacenados en los mismos archivos con una vista distinta.

1.8 Modelos de datos

En los ambientes de desarrollo existen lenguajes de programación estructurados, orientados a aspectos, orientados a objetos, etc.; donde cada uno brinda conceptos y ventajas en determinados tipos de aplicaciones.

En los ambientes de base de datos también existen modelos de datos para almacenar los datos de la empresa, algunos que históricamente dieron las bases para el desarrollo de los demás, otros que están actualmente en uso y los que están en proceso de investigación.

Un modelo de datos brinda distintos conceptos y permite definir las reglas y las estructuras para el almacenamiento de datos para, después, manipularlos.

Los modelos de datos se clasifican de diversas formas. Sin embargo, en la literatura especializada, existe cierta coincidencia en utilizar —para su taxonomía— los conceptos que conforman la base de cada uno de ellos. Por ejemplo: hay modelos de datos basados en objetos y otros en registros; también el modelo físico de datos que apunta al almacenamiento de los datos en un nivel bajo de abstracción, relacionado con el formato y con el camino de acceso a los datos.

En los modelos de datos basados en registros, los datos se estructuran en un conjunto de campos que conforman un registro. Tal es el caso de un registro “Estudiante” conformado por campos como legajo, apellido, nombres, fecha y lugar de nacimiento.

En el caso de los modelos basados en objetos, se han aprovechado los conceptos utilizados en el paradigma de programación orientada a objetos, como sucede en las áreas de ingeniería de software. Solo que, en las bases de datos orientadas a objetos, existe la posibilidad de crear objetos para que tengan un almacenamiento que persista en el tiempo y para que puedan ser utilizados.

1.8.1 Modelo de datos orientado a objetos

Este modelo nace con el objetivo de permitir el almacenamiento de datos para aplicaciones complejas que —como dicen Elisa Bertino y Lorenzo Martino en *Sistemas de bases de datos orientadas a objetos*— no se orientan a las del área comercial y administrativa, sino a las de las ingenierías, tales como CAD/CAM, CASE, CIM, sistemas multimediales y sistemas de gestión de imágenes, en las que la estructura de los datos es compleja y las operaciones se definen en función de la necesidad de las aplicaciones.

El grupo de gestión de bases de datos orientadas a objetos (ODMG, *Object Database Management Group*) —conformado por usuarios y productores de sistemas de gestión de bases de datos orientados a objetos (OODBMS, *Object-Oriented Database Management System*)— es el encargado de estandarizar el modelo de datos, el lenguaje de definición de objetos (ODL, *Object Definition Language*) y el lenguaje de consulta de objetos (OQL, *Object Query Language*).

Los datos se almacenan de manera persistente en objetos, que se identifican, unívocamente, por un OID (identificador de objeto), generado por el sistema.

Los objetos se definen con una estructura (OID, Constructor_Tipo, Estado) en la que:

- OID: identificador único en el sistema; incluso, cuando se elimina un objeto, es conveniente que el OID no se reutilice.



Un modelo de datos brinda distintos conceptos y permite definir las reglas y las estructuras para el almacenamiento de datos para, luego, manipularlos.



Los datos se almacenan de manera persistente en objetos, que se identifican, unívocamente, por un OID, generado por el sistema.

- Constructor_Tipo: es la definición de la construcción del estado del objeto.
- Estado: se define a través del Constructor_Tipo.

Por ejemplo: si el Constructor_Tipo es de tupla, significa que el estado tendrá la estructura, donde es un nombre de atributo y es un OID.

Si el Constructor_Tipo es átomo, significa que el estado será un valor.

De esta manera, el Constructor también puede ser un conjunto, una lista, una bolsa y un *array* y determina que el estado se refiera a OID de objetos. Solo el átomo se puede referir a un valor, aunque éste puede, también, referirse a otro objeto mediante su OID.

Con estos constructores, se producen anidamientos y se logra una estructura de alta complejidad, además de la posibilidad de incorporar conceptos propios de la programación orientada a objetos: como clase, herencia, polimorfismo, etcétera.

La definición en ODL de una clase “Personas” se puede expresar de la siguiente manera:

```
Class Personas
{
  attribute string apellido,
  attribute string nombres,
  attribute enum PosiblesSexo {Masculino, Femenino} sexo,
  attribute struct Domicilio
    {string calle, short numero, string ciudad,
  attribute date fecha_nac;
  short edad();
};
```

Para consultar la base de datos, se utiliza el lenguaje OQL propuesto por ODMG. Las sentencias de este lenguaje son similares a las de SQL y pueden ser insertadas en programas desarrollados con lenguajes de la programación orientada a objetos, como en el caso de C++ o Java. El resultado es un conjunto de objetos que pueden ser manipulados directamente en esos lenguajes y otros que sigan el paradigma.

Una consulta OQL puede ser:

```
SELECT p.apellido
FROM p in Personas
WHERE p.sexo="Masculino"
```

En esta expresión, aparece “p” como variable iteradora que asume los valores de cada objeto y la consulta devuelve una bolsa *string*, porque es una colección de apellidos que pueden repetirse.

Entre las ventajas de este modelo, se pueden distinguir: la posibilidad de manipular objetos complejos con buen rendimiento, la integración de la persistencia de datos a la programación orientada a objetos y un menor costo y esfuerzo en el desarrollo de las aplicaciones y en el mantenimiento.

La aparición de este tipo de sistemas de gestión de bases de datos exigió a los RDBMS (sistemas de gestión de bases de datos relacionales), que incorporaran objetos en sus estructuras y flexibilizaran las posibilidades de almacenamiento; de ello, surge el concepto actual de sistemas de gestión de bases de datos objeto-relacionales.

En la actualidad, se conocen prototipos y productos de gestión de bases de datos orientadas a objetos como, por ejemplo:

- Jasmine: tecnología orientada a objetos que permite tratar datos multimedia y complejas informaciones relacionadas. Se trata de un producto Java que se puede utilizar en entornos Intranet, Internet, Cliente/Servidor y a través de cualquier navegador.
- GemStone: introducido en 1987, es el ODBMS comercial con más tiempo en el mercado. Soporta acceso concurrente de múltiples lenguajes, que incluye al Smalltalk-80, Smalltalk/V, C++ y C.
- Versant Object Database (V/OD): ofrece características importantes a los desarrolladores C++, Java o NET, el apoyo a la concurrencia masiva y a grandes conjuntos de datos.

Éste es un tema tan interesante como extenso; por ello, se dejará su tratamiento para otro capítulo y se empezará con la explicación de las bases de conceptos para llegar al modelo relacional.

1.8.2 Modelos de datos basados en registros

Estos modelos de datos tienen sus inicios con los modelos de red y jerárquicos, hoy obsoletos desde el punto de vista tecnológico.

Ambos modelos, red y jerárquicos, compartían el concepto de registro de datos y los enlaces físicos entre ellos, pero diferían en sus restricciones y en su representación.

Posteriormente, surgió el Modelo Relacional, donde se mantiene el concepto de registro de datos y las relaciones entre las entidades del mundo real son lógicas, dando origen a entidades que permiten establecer relaciones, como también a otros temas propios del modelo. Por la importancia que tiene el tema en sí mismo, se decidió dedicar otros capítulos al Modelo Relacional.

1.8.2.1 Características del modelo de datos en red

Los datos se representan como una colección de registros y las relaciones entre ellos mediante enlaces, que se implementan como campos que almacenan punteros.

Cada registro es una colección de campos y cada campo almacena un dato.

Gráficamente, se los representaba con un diagrama, en el que la estructura de datos estaba formada por cajas y líneas. Las cajas eran los registros y las líneas indicaban sus relaciones.



Los datos se representan como una colección de registros y las relaciones entre ellos mediante enlaces, que se implementan como campos que almacenan punteros.

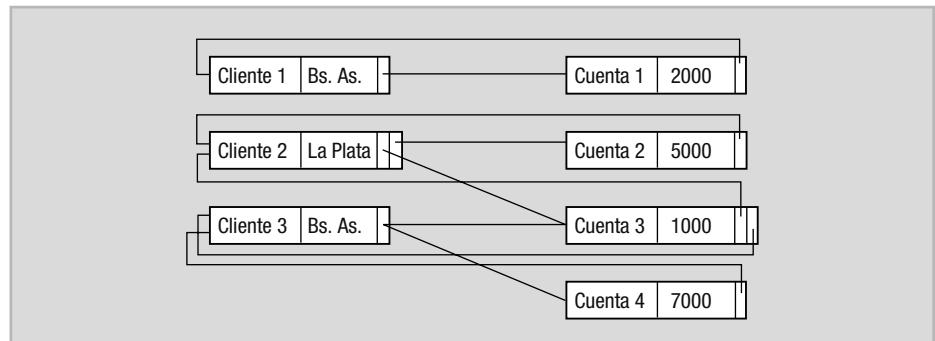


Fig. 1.3 Modelo de datos en red.

La Fig. 1.3 representa a tres registros de clientes y a cuatro registros de cuentas bancarias donde:

- Cliente 1 posee una sola cuenta con un saldo de \$2000. Se observa que este registro tiene un campo para un puntero del Cliente a la Cuenta 1 y, en el registro Cuenta 1, se visualiza un campo con un puntero hacia el Cliente 1.
- Cliente 2 tiene una Cuenta 2 a su nombre y comparte la Cuenta 3 con el Cliente 3. En este caso, en Cliente 2 hay dos campos desde donde parten dos punteros.
- Cliente 3 comparte la Cuenta 3 con el Cliente 2 y, además, posee su propia cuenta que es la 4.

Se ve que en el registro de la Cuenta 3 hay dos campos, porque tiene un puntero hacia el Cliente 2 y otro, hacia el 3.

Los campos que almacenaban punteros hacían que los registros fueran de longitud variable, porque en algunos casos se necesitaba solo uno y en otros, más.

La flexibilidad del modelo hizo que la estructura de datos fuera compleja, ya que no había restricciones, en la que se puede observar cierta estructura Padre-Hijo.

Las acciones de manipulación de datos en la programación implicaban órdenes que significaban movimientos físicos. Ejemplo:

- *Find*: localiza un registro, según alguna condición y el puntero actual, en el archivo, queda apuntando a ese registro.
- *Get*: copia registro señalado por el puntero actual en el área de trabajo-*buffer* de memoria.
- *Store*: crea un registro nuevo, en un archivo, con valores de datos ubicados en la memoria que maneja el usuario.
- *Modify*: modifica contenido. Implica encontrarlo, subirlo a la memoria y, solo ahí, modificarlo.
- *Erase*: elimina registro, con su búsqueda previa.
- *Connect*: conecta un registro con un conjunto de registros de un archivo.
- *Disconnect*: desconecta un registro de un conjunto archivo.

- *Reconnect*: mueve un registro de un conjunto a otro. Por ejemplo: cambia una cuenta de una sucursal a otra.

Al crear los conjuntos archivos, se debe indicar qué hacer ante las siguientes acciones:

- Inserción: si la inserción es manual, debe existir en el programa una instrucción *connect* para enlazar el nuevo registro y, si no, puede ser automática, donde al ejecutar *store* el registro se almacena en el conjunto y se enlaza.
- Eliminación: depende si se definió como;
 - Opcional, en el cual pueden existir registros sin conexión al conjunto (elimina padre y desconecta todos los hijos).
 - Fijo, en el cual no puede haber registros de miembros sin conexión al grupo y, además, no pueden reconectarse con otra ocurrencia de un conjunto (elimina padre y elimina todos los hijos).
 - Obligatorio, tampoco puede haber registros sin conexión al grupo, pero sí pueden moverse de un conjunto a otro (no deja eliminar el registro).
- Ordenación: en el conjunto “archivo” debe definirse qué se hará al añadir un nuevo registro. Las opciones son que, al insertarlo, pase a alguna de estas posiciones en el conjunto:
 - Primero del conjunto.
 - Último.
 - Siguiente al registro actual del conjunto.
 - Anterior al registro actual.
 - Arbitrario, según determinación del sistema de gestión.
 - Ordenado según algún criterio y el sistema de gestión debe mantener el orden.

1.8.2.2 Características del modelo de datos jerárquico

Este modelo se puede considerar una implementación particular del modelo en red, en el cual la estructura física también se basa en registros y en punteros, pero con forma de árbol invertido y con restricciones en las relaciones.

Las relaciones se representan mediante enlaces implementados como campos que almacenan punteros.

Los grafos se organizan como árboles con un nodo raíz.

Un registro es una colección de campos y cada uno de esos campos almacena un dato. A la colección de registros, se la denomina tipo de registro, que son los nodos en el árbol.

Las relaciones que se mantienen entre tipos de registros son del tipo 1: N, en la que se distinguen un registro “Padre” y n registros Hijos.

En el diagrama jerárquico, se muestra la estructura de árbol, con cajas que representan los tipos de registros y líneas que representan las relaciones “Padre-Hijo”.

Los niveles del árbol no tienen limitación y el nodo raíz está en el nivel 0 (cero).



Un registro es una colección de campos y cada uno de esos campos almacena un dato.

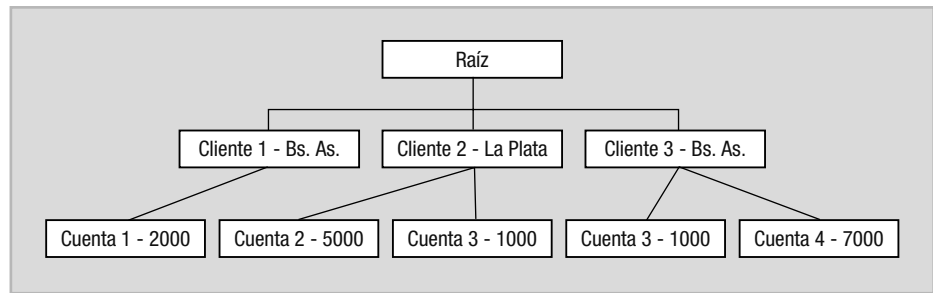


Fig. 1.4 Modelo de datos jerárquico.

Algunas restricciones del modelo son:

- El nodo raíz no participa como hijo en ninguna relación y los tipos de registros son siempre hijos de un solo tipo de registros.
- Un nodo padre puede tener un número ilimitado de hijos, pero un nodo hijo puede y debe tener solo un nodo padre.
- Si un tipo de registro “Padre” posee más de un tipo de registro “Hijo”, los tipos de registros “Hijo” estarán ordenados de izquierda a derecha.
- Se puede eliminar un registro “Hijo” y el registro “Padre” puede quedar con 0 (cero) hijos, pero si se desea eliminar en el registro “Padre” no puede quedar un registro “Hijo” sin padre, por lo que se elimina el “Padre” y todos los hijos asociados.

Para que un registro “Hijo” tenga más de un registro “Padre”, debe duplicarse con la consiguiente redundancia de datos. Algunas implementaciones evitan esto mediante uso de registros virtuales con punteros.

Ese tipo de situaciones muestran una desventaja del modelo jerárquico: la poca flexibilidad que posee para representar estructuras que, en la vida real, hacen falta. Que dos o más clientes de un banco compartan una cuenta es muy común, como también en una estructura de productos existen productos que son piezas de otro producto (estructura recursiva de datos).

Algunas acciones que se pueden mencionar en la manipulación de datos de este modelo:

- *Get*: localiza y copia un registro al área de trabajo las condiciones de búsqueda se agregan con la cláusula WHERE. Con un ciclo, se podía recorrer toda una rama del árbol, leyendo todos los registros al recorrerla, sin que importe la altura del árbol.
- *Insert*: añade un registro al árbol. Primero se localiza a su registro “Padre” y luego, se emite la orden de inserción.
- *Replace*: modifica uno o más campos de un registro que, primeramente, se ubicará como el registro actual de la base de datos.
- *Delete*: elimina un registro de la base de datos que esté como registro actual.